

INTRODUZIONE

Nome progetto: *Brent Tracker*

Descrizione del sistema: Brent Tracker è un sistema software progettato per fornire accesso autenticato a dati relativi al prezzo del petrolio Brent, integrando una sorgente dati esterna con una persistenza locale e un'architettura stratificata.

Obiettivi e Funzionalità MVP (Minimum Viable Product): Il sistema espone un'interfaccia API che garantisce le funzionalità fondamentali. L'utente ha la possibilità di registrarsi ed effettuare l'accesso in modo sicuro. L'utente può recuperare le quotazioni del Brent e consultare l'archivio delle quotazioni del Brent richieste da tutti gli utenti. Inoltre, il sistema permette all'utente di configurare alert di prezzo (creazione, eliminazione, attivazione/disattivazione).

ANALISI dei REQUISITI

Requisiti funzionali

RF1	Gestione Utenti: registrazione di un nuovo account utente e relativa autenticazione (login).
RF2	Monitoraggio Prezzi: consultazione del prezzo corrente del petrolio Brent e aggiornamento automatico dei dati.
RF3	Storico Dati: Salvataggio e visualizzazione dello storico dei prezzi registrati.
RF4	Gestione Alert: Creazione, visualizzazione, eliminazione e gestione dello stato (attivazione/disattivazione) degli alert personalizzati sulle soglie di prezzo.

Requisiti non funzionali

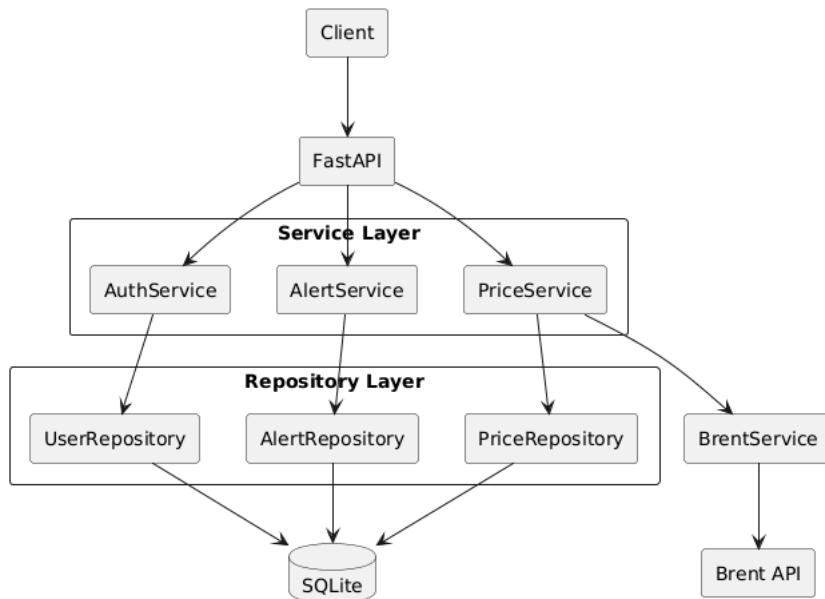
RNF1	Sicurezza: il sistema deve garantire l'accesso sicuro ai dati. Le password degli utenti non devono mai essere salvate in chiaro, ma protette tramite algoritmi di hashing sicuri. Le chiamate API per le funzionalità riservate devono essere protette da meccanismi di autenticazione.
RNF2	Prestazioni: il sistema deve garantire tempi di risposta rapidi per le interrogazioni API, assicurando che le operazioni di lettura e scrittura sul database locale non introducano colli di bottiglia.
RNF3	Manutenibilità e testabilità: il codice deve essere strutturato in moduli indipendenti (architettura a livelli) per facilitare modifiche future. Il sistema deve essere altamente testabile, garantendo una copertura dei test automatici (Test Coverage) uguale o superiore all'80%.
RNF4	Portabilità: l'applicazione deve essere agnostica rispetto al sistema operativo sottostante e facilmente replicabile in ambienti diversi. Questo requisito è soddisfatto tramite la containerizzazione del sistema con Docker.
RNF5	Affidabilità: il backend deve essere resiliente. In caso di disservizio, lentezza o mancata risposta da parte del provider esterno dell'API Brent, il sistema deve gestire l'errore senza andare in crash, restituendo messaggi di errore chiari all'utente.
RNF6	Interoperabilità: il sistema deve poter comunicare in modo standardizzato con eventuali client futuri (come un frontend minimale). Per questo, deve esporre un'interfaccia API REST.

ARCHITETTURA del SISTEMA

Architettura

Il sistema si basa su un'architettura a strati composta da API layer, service layer e repository layer. Tale separazione permette di isolare la logica applicativa dalla persistenza dei dati e dai servizi esterni.

Il provider esterno è isolato: in caso di future interruzioni di servizio basterà aggiornare l'API utilizzata dal modulo BrentService.



IMPLEMENTAZIONE

L'implementazione del sistema è stata realizzata in Python utilizzando il framework FastAPI.

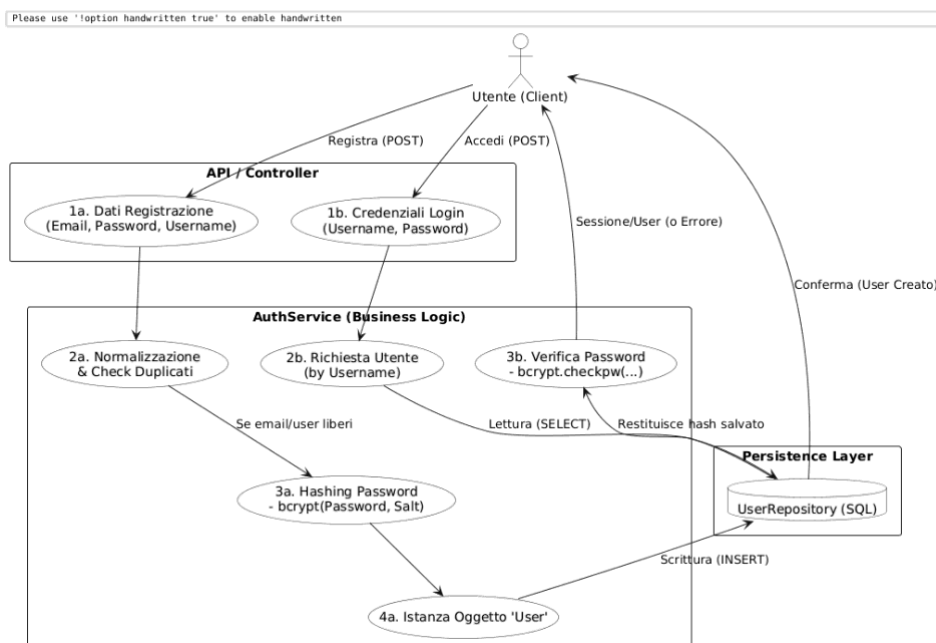
La struttura applicativa è suddivisa principalmente in router, servizi e repository. I router espongono le API REST e si occupano della gestione delle richieste HTTP, mentre la logica applicativa è implementata all'interno dei service. L'accesso al database è invece demandato ai repository, evitando di accoppiare direttamente la logica applicativa ai dettagli di persistenza.

Gestione degli utenti e autenticazione

La gestione degli utenti è stata implementata attraverso un sistema di registrazione e autenticazione. Durante la registrazione vengono validati i dati ricevuti e viene verificata l'eventuale presenza di un utente già registrato con gli stessi dati identificativi. La password non viene memorizzata in chiaro, ma sottoposta a hashing prima della persistenza nel database.

Il login verifica le credenziali fornite dall'utente e, in caso di autenticazione corretta, genera un token di accesso Bearer.

Flusso dei Dati: Registrazione vs Login



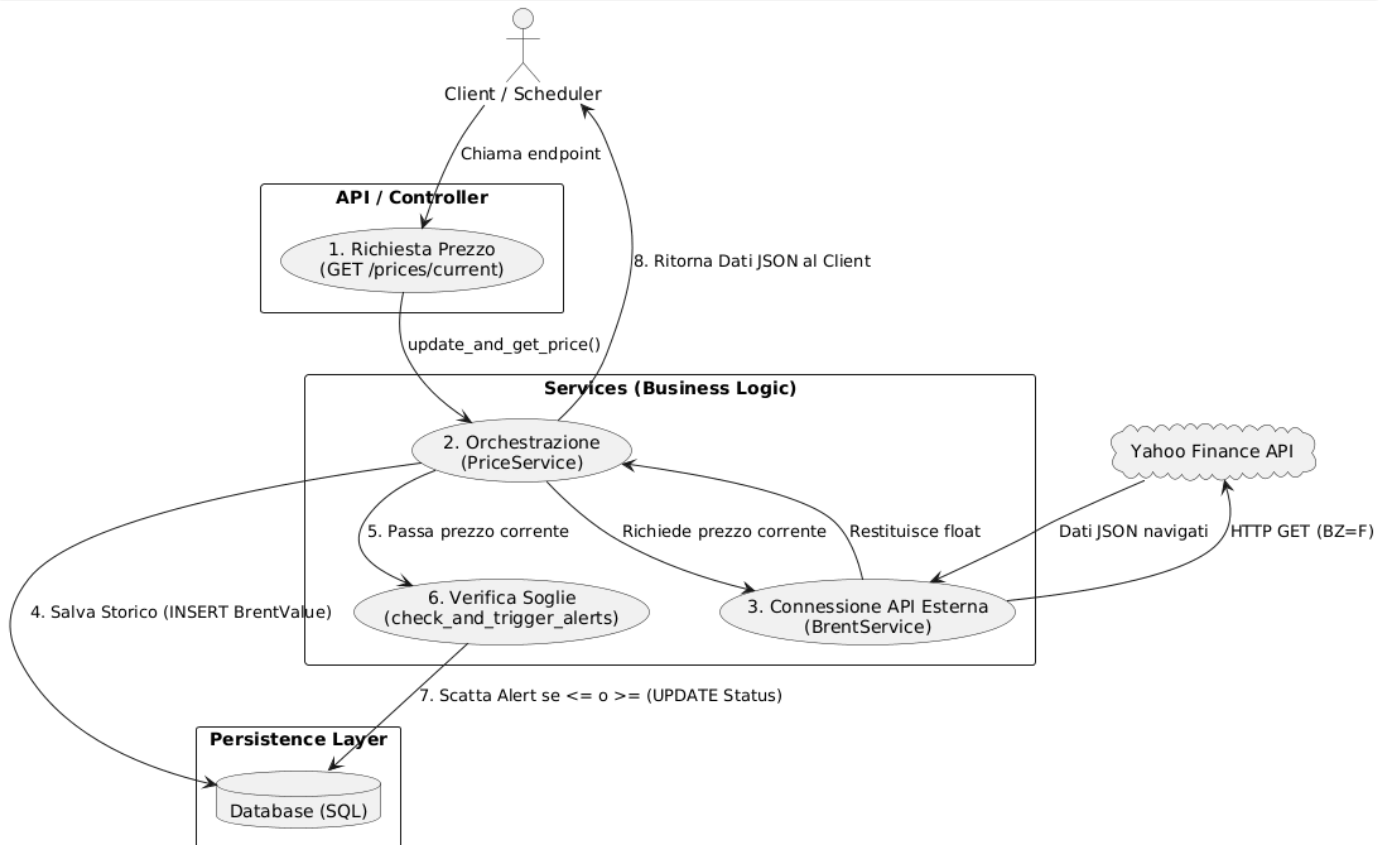
Gestione dei dati relativi al Brent

Il sistema integra una sorgente esterna per il recupero del prezzo del petrolio Brent. La comunicazione con il servizio esterno è incapsulata all'interno di un componente dedicato, il Brent Service, che si occupa del recupero e dell'elaborazione dei dati ricevuti.

I dati ottenuti vengono successivamente inseriti nel database locale tramite il livello repository. Questa scelta consente di mantenere uno storico dei prezzi e di separare la gestione della sorgente esterna dalla logica di accesso ai dati persistiti. Le API messe a disposizione permettono all'utente di ottenere il prezzo più recente disponibile con controllo alert annesso e di consultare lo storico dei dati memorizzati.

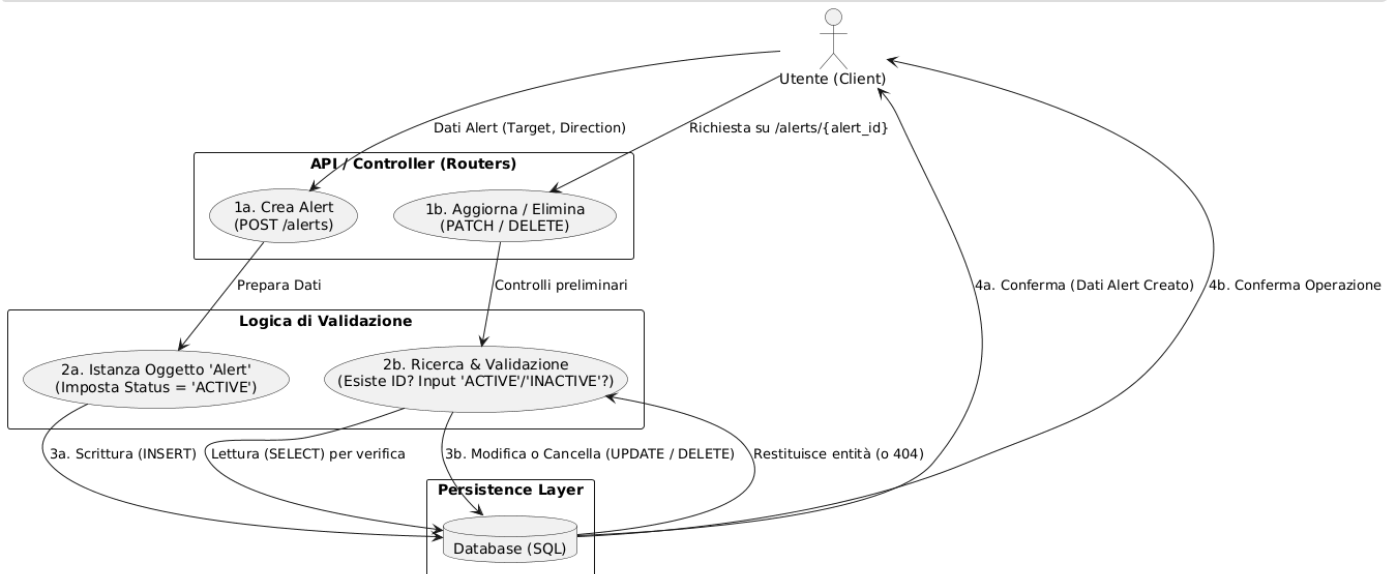
Flusso dei Dati: Recupero Prezzo e Controllo Alert

Please use '!option handwritten true' to enable handwritten



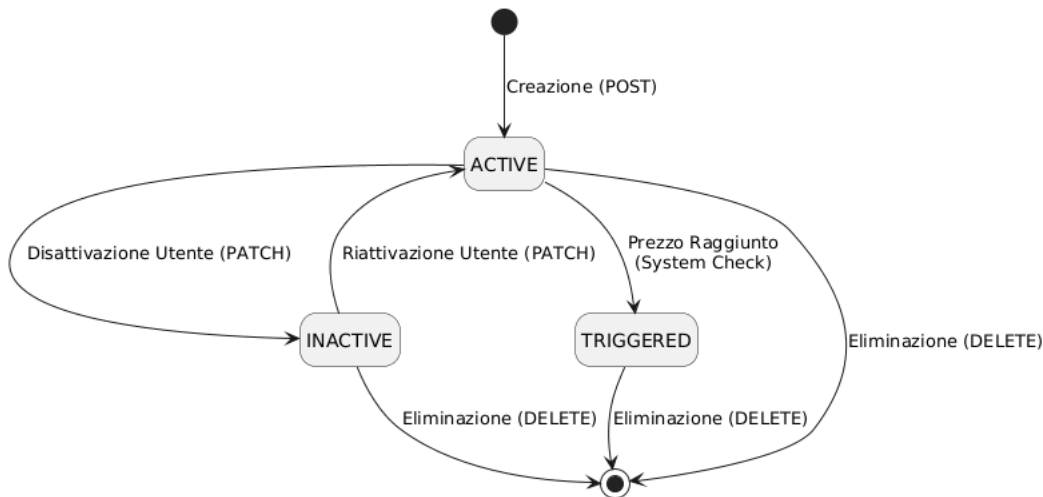
Flusso dei Dati: Gestione degli Alert (CRUD)

Please use '!option handwritten true' to enable handwritten



Macchina a Stati dell'Entità "Alert"

Please use 'loption handwritten true' to enable handwritten



Validazione e gestione degli errori

La validazione dei dati in ingresso è stata realizzata mediante i modelli Pydantic. In questo modo i dati ricevuti dalle API vengono controllati prima di essere elaborati dalla logica applicativa.

Gli errori applicativi e le condizioni anomale, come il tentativo di registrare un utente già esistente o l'accesso con credenziali non valide, vengono gestiti restituendo appropriati codici di stato HTTP e messaggi descrittivi.

L'implementazione è stata inoltre progettata per facilitare il testing delle singole componenti. In particolare, le dipendenze tra i vari livelli e la comunicazione con il servizio esterno possono essere sostituite o simulate durante l'esecuzione dei test.

STRATEGIA DI TESTING

La strategia di testing adottata combina test unitari, property-based testing e test di integrazione, con l'obiettivo di verificare sia il comportamento delle singole componenti sia il corretto funzionamento dei principali flussi applicativi.

I test unitari relativi all'autenticazione verificano innanzitutto il corretto funzionamento dell'hashing delle password. In particolare, viene controllato che la password non venga memorizzata in chiaro e che una password corretta possa essere verificata a partire dal relativo hash. È stato inoltre testato il processo di registrazione, verificando la normalizzazione dei dati, la creazione dell'utente e il corretto utilizzo del repository. Sono stati considerati anche casi di errore, come il tentativo di registrare un username già esistente. Infine, il processo di autenticazione è stato verificato sia con credenziali valide sia con una password errata, controllando che in quest'ultimo caso l'autenticazione fallisca.

Per il componente responsabile del recupero del prezzo Brent è stato utilizzato un mock della risposta HTTP esterna. In questo modo è stato possibile verificare che il servizio estragga correttamente il prezzo dalla risposta ricevuta e che vengano eseguite le operazioni previste sulla risposta HTTP, senza dipendere realmente dal servizio esterno durante i test. Il servizio di gestione dei prezzi è stato testato verificando il corretto recupero e ordinamento cronologico dello storico dei prezzi. È stato inoltre verificato il flusso di aggiornamento del prezzo: il servizio deve recuperare il valore corrente, salvarlo tramite il repository e attivare il controllo degli eventuali alert associati al nuovo prezzo. I mock permettono di controllare anche le interazioni tra i diversi componenti e di verificare che le dipendenze vengano utilizzate correttamente.

È stato inoltre adottato il property-based testing tramite Hypothesis. In particolare, sono state generate automaticamente password valide appartenenti all'alfabeto previsto, diversi valori di prezzo e liste di valori storici. Questa tecnica permette di verificare proprietà generali del sistema su numerosi input differenti, riducendo la dipendenza da pochi casi di test scelti manualmente.

Infine, è stato realizzato un test di integrazione end-to-end per verificare il corretto funzionamento di un flusso applicativo completo. Il test simula la registrazione di un utente, il successivo login, il recupero del prezzo corrente del Brent, la persistenza del prezzo nello storico e la creazione di un alert. Per garantire l'isolamento del test, viene utilizzato un database SQLite temporaneo e la chiamata al servizio esterno del Brent viene sostituita con un valore simulato. In questo modo è possibile verificare l'interazione tra API, servizi e persistenza senza dipendere da componenti esterne.

CONTAINERIZZAZIONE

La containerizzazione dell'applicazione è stata realizzata tramite Docker utilizzando come immagine di base python:3.13-slim. La variante slim è stata scelta per ridurre le dimensioni dell'immagine e limitare la presenza di componenti non necessari all'esecuzione dell'applicazione.

Il file pyproject.toml e il lockfile uv.lock vengono copiati inizialmente all'interno del container e utilizzati per installare le dipendenze tramite uv. L'opzione --frozen garantisce che vengano utilizzate esattamente le versioni definite nel lockfile, rendendo l'ambiente di esecuzione riproducibile. L'opzione --no-dev evita invece l'installazione delle dipendenze utilizzate esclusivamente durante lo sviluppo e il testing.

Successivamente viene copiato il codice dell'applicazione nel container. La porta 8000 viene esposta per rendere disponibile il servizio FastAPI. L'applicazione viene infine avviata tramite Uvicorn, configurato per ascoltare su 0.0.0.0, così da consentire l'accesso al servizio dall'esterno del container.

La scelta di Docker permette di isolare l'applicazione dal sistema ospite e di garantire un ambiente di esecuzione coerente e riproducibile, facilitando il deployment e l'integrazione con la pipeline CI/CD.

CI/CD

La pipeline CI/CD è stata implementata tramite GitHub Actions ed è configurata per essere eseguita automaticamente in seguito a un push sul branch main o all'apertura di una pull request verso lo stesso branch. Questa scelta permette di verificare automaticamente le modifiche prima della loro integrazione nel ramo principale.

Il job viene eseguito su un ambiente Ubuntu e utilizza Python 3.13, in modo da mantenere coerenza con l'ambiente di sviluppo e con il container Docker dell'applicazione. Dopo il checkout del repository e l'installazione del package manager uv, vengono installate le dipendenze definite dal progetto.

La pipeline esegue quindi automaticamente la suite di test con pytest e calcola la code coverage tramite pytest-cov. Oltre al report testuale, viene generato un report HTML che viene caricato come artifact della pipeline. Questa scelta permette di conservare e consultare il risultato dell'analisi della copertura del codice.

In seguito, la pipeline verifica la corretta costruzione dell'immagine Docker. Dopo questa fase viene effettuato il login a Docker Hub utilizzando credenziali memorizzate come GitHub Secrets, evitando di inserire dati sensibili direttamente nel codice della pipeline.

Infine, l'immagine Docker viene pubblicata automaticamente su Docker Hub con il tag latest. In questo modo ogni modifica integrata nel branch principale, dopo il superamento dei controlli previsti, può produrre una nuova immagine del sistema pronta per essere utilizzata nel deployment.

La pipeline automatizza quindi le principali fasi di verifica e distribuzione del progetto, riducendo le operazioni manuali e garantendo un processo di integrazione e rilascio riproducibile.